Java basics



Image: “Juice Fruit Juice Green Juice”
From http://all-free-download.com/ Public domain

Variables in Java

Unlike Python, Java is **statically typed**

So types of all variables must be declared

- Local variables:

```
int i;
```

- Class fields:

```
class C { boolean b; }
```

- Method parameter and return values

```
public float getValue (long l) { … }
```

Static typing in practice

Variables can only be given values that are compatible with their type

```
int i;
i = 5;
i = 5.0;
i = “hello”
```

Variables cannot be redeclared with a different type

```
String s = “hello”;
int s;
```

List of Java primitive types

Type	Description	Min value	Max value	Default value
`byte`	8-bit signed integer value	-128	127	0
`short`	16-bit signed integer value	-32,768	32,767	0
`int`	32-bit signed integer value	-2^{31}	$2^{31} - 1$	0
`long`	64-bit signed integer value	-2^{63}	$2^{63} - 1$	0L
`float`	32-bit single precision IEEE 754 floating point value	2^{-149}	$(2-2^{-23})\cdot 2^{127}$	+0.0F
`double`	64-bit double precision IEEE 754 floating point value	2^{-1074}	$(2-2^{-52})\cdot 2^{1023}$	+0.0
`boolean`	Boolean value (true/false)	n/a	n/a	false
`char`	16-bit Unicode character value (use single quotes – e.g., `'c'`)	\u0000 (0)	\uffff (65,535)	\u0000

What about strings?

Technically, Java strings are of type `java.lang.String` which is an object (not primitive)

However, the Java language has special support for strings

E.g., you can create a new one by using double quotes

```
String s = "This is a string";
```

Also, they are **immutable** (will be explained later in the course)

So you will probably often tend to think of them as primitive – just don't forget that they are not!

Useful operators for primitive types

Assignment = (single equals sign)

```
int i = 5;
```

Equality comparison == (two equals signs), != (not equal to)

```
if (i == 5) { … }
if (j != 5) { … }
```

Relational operators <, >, <=, >=

Boolean combinations && (and), || (or)

*Special considerations for String (because it's an **object**) – stay tuned*

Type conversions

Remember, Java is a **statically typed** language

Any variable can only hold values of a single, specific type

To store a value of type t_1 in a variable of type t_2, the value must be **converted to** t_2 **before** the assignment occurs

This is unlike Python where you can do something like this (and more!)

```
foo = 5
foo = True
foo = “bar”
foo = 0.5
```

Implicit (widening) conversions

Sometimes, type conversion can happen automatically with no extra source code

Generally, these are *widening* conversions – little or no information lost

`byte` to `long`

`long` to `double` *// potential loss of precision*

…

Precision may be lost, but the **magnitude** of the numeric value is preserved

```
int i = 5000;
long l = i;
double d = l;
```

Explicit (narrowing) conversions – casting

Some conversions would result in significant potential information loss

- `double` to `float`
- `int` to `short`
- …

These **narrowing** conversions must be made explicit in source code using *casting*: specify the target type in round brackets

```
int i = 1025;
byte b = (byte)i;
// b now has value 1
```

How does narrowing work?

`double` to `float`: loss of precision, plus …

- Out-of-range values become infinite
- Some non-zero values may become zero

`double/float` to `long/int/short/byte/char`

- Round-to-zero
- Out-of-range becomes infinity

Integer type to "smaller" integer type (e.g., `long` to `int`, `int` to `byte`, …)

- Discard all but *n* lowest-order bits
- In practice: value *mod* max

`String` ↔ primitive types

Converting from `String`

Use `parseXXX` methods of primitive wrapper classes

E.g., `int i = Integer.parseInt("42");`

Converting to `String`:

1. Just concatenate (+) with an existing `String` value
2. Use `String.valueOf(…)`

A note on integer division

Most mathematical operators (+, -, *, …) work as you would expect in Java

Function of division operator "/" depends on type of the two arguments

- If **both** are integers (`int, long, short, byte, char`), then it does **integer division**
- If **either** is floating-point (`float, double`), then it does **floating point division**

Example:

`7.0/4.0`	returns **`1.75`** (same result for `7.0/4` and `7/4.0`)
`7/4`	returns **`1`**

General rule: integer division throws away the remainder (so `99/100 == 0`)

Blocks and scope

Block of statements is enclosed in curly braces { }

Usually all statements in a block have same indentation level

- Unlike Python, this is not mandatory – but strongly recommended!

A variable is in scope (accessible) …

- From the point of declaration …
- To the end of the enclosing block
- AND NOT AFTER!

```
int i = 5;
int j = 10;

{
        int i = 20;
        j = 3;
        int k = i + j;

        System.out.println(i);
        System.out.println(j);
        System.out.println(k);

}

System.out.println(i);
System.out.println(j);
System.out.println(k);
```

If statements in Java

Syntax is straightforward:

If (*condition1)*
    block1
else if (*condition2)*
    block2
// …
else
    blockN

The Java *for* loop

Syntax is similar to C, C++, C#, JavaScript, …

Contains three clauses

- Initialisation
- Termination condition check
- Variable update

Termination condition is checked before loop is executed

Loop variable is local to the loop – can't be used before or after

- It's a normal variable though and can be updated within the loop!

```
for (int i = 0; i < 10; i++) {
  doSomething();
}
```

The *while* loop

Similar to – but more general than – the *for* loop

Termination condition is checked each time before the loop is run

- What if ***condition*** is false at the start?
- What if ***condition*** is never false?

Loop variable is available before and after the loop (if you do a *for*-like loop)

```
while (condition) {
  doSomething();
}
```

```
int i = 0;
while (i<10) {
   doSomething();
   i++;
}
```

Advanced loop control flow

for and *while* loops both have **termination conditions**

```
for (…; …; i<10)     while (!valid)
```

Condition is checked, and if it passes, entire loop is executed before condition is checked again

But what if you want to do something different with your loop?

- Skip the rest of a single loop execution – `continue`
- Break out of the loop immediately – `break`

switch statements

IF STATEMENTS

Evaluates any `boolean` expression

Conditionally executes one of two statements or blocks

SWITCH STATEMENTS

Evaluates an integer or `String` expression

- Integer: `byte`, `short`, `int`, `long`, `char`, `enum`

Conditionally executes one or more of a list of `case` blocks

When to choose one or the other? Depends on (1) readability, (2) the nature of the expression being evaluated (complex tests require `if`)

Details of how `switch` works

Switch statement evaluates its expression ...

... and then executes **all** statements following the matching `case` label

... until it encounters a `break` statement

```
int vs = 0, cs = 0;

for (int i=0; i<s.length(); i++) {
    char c = s.charAt(i);
    switch (c) {

        case 'a':
        case 'e':
        // …
            vs++;
            break;

        case 'b':
        case 'c':
        // …
            cs++;
            break;
        default:
           System.out.println ("other");
    }
}

System.out.println("Vowels:" + vs + "Consonants:" + cs);
```

Declaring a method in Java: Syntax

A method declaration has six components (in order):

1. Access modifier(s) (zero or more – details later)
2. Return type (`void` if it does not return a value)
3. Method name (conventionally beginning with a verb)
4. Parameter list in parentheses – comma delimited list of input parameters, preceded by data type, enclosed in parens. No parameters – empty parens.
5. An exception list (more on this later)
6. The method body, enclosed in braces { }

Method signature

```
public double calculateAnswer(double wingSpan, int numberOfEngines,
                              double length, double grossTons) {
    //do the calculation here
}
```

Calling a method

Use the name

Specify the parameters in parentheses

Store the result in a variable of the correct type

(Not necessary if method is **void**; optional other times)

```
public double calculateAnswer(double wingSpan, int numberOfEngines,
                              double length, double grossTons) {
    //do the calculation here
}

double result = calculateAnswer(10, 4, 100, 1000);
```

More on methods

To return from a method, use **return** statement

Multiple **return** statements are possible!

But be sure to return the correct type

It is possible to have two methods with the same name as long as they have different parameter lists (this is called **overloading**)

```
double doSomething(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

```
double doSomething(double a) {
    return 0 – a;
}
```

```
int doSomething(String s) {
    return s.length();
}
```

Declaring an array

Each array has …

A **type** – the type of the individual elements in the array

A **dimension** – the dimensionality

Examples:

`int[]` – a one-dimensional array of integers

`String[][]` – a two-dimensional array of Strings

Variable declaration (method parameter, local variable, class field):

```
String[] args
```

(`String args[]` also valid – hang-over from C language style)

Initializing an array

Declaring an array **does not** …

- Reserve space for the array elements
- Specify the length of the array

Technically, it only creates a **reference** to point to the array

Two options for creating an array:

- Using **`new`** and giving an explicit size
- Using an “initializer” to give the initial values (shortcut syntax)

Initializing an array with `new`

```
int[] values = new int[10];
```

Allocates enough space to store the specified number of elements

Fills the array with default values

- 0 for numeric types
- Null for object types

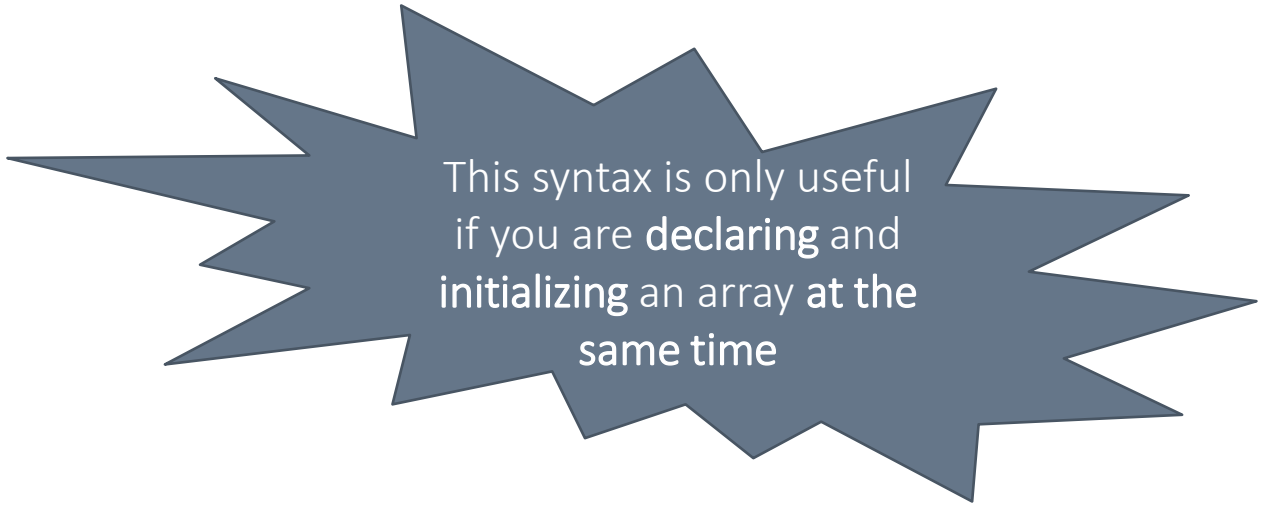
Initializing an array directly

Put a comma-separated list of elements between curly brackets

```
int[] values = { 10, 20, 30, 40, 50, };
```

Array length is determined from the length of the input list

(Nice feature: you can put a comma at the end of the list)



Accessing array elements

Use square brackets around index

```
int value = values[3];
```

Subscripts start at zero (as in Python – unlike other languages such as Fortran, COBOL, Matlab)

For multidimensional arrays, use multiple sets of brackets:

```
String[][] strings = // …;
System.out.println( strings[i][j] );
```

Array length

An array's length **cannot** be changed after it is initialized

Any attempt to use an out-of-range index will result in an error

Array length can be accessed through built-in field `length` (note: **not** a method)

```
String[] strings = { "Each", "peach", "pear", "plum", };
System.out.println (strings.length);
// Prints out 4
```

Iterating through an array

```
String[] fruits = new String[] { "apple", "banana", "cherry" ];
```

Standard Java idiom: **`for`** loop

```
for (int i = 0; i < fruits.length; i++) {
    System.out.println(fruits[i]);
}
```

More efficient option: ***for-each*** loop (since Java 1.5) – similar to Python's **for** loop

```
for (String fruit : fruits) {
    System.out.println (fruit);
}
```

Java
Objects and inheritance



Image: “Juicy” by jan buchholtz
https://flic.kr/p/ojAtFi CC-BY-ND-2.0

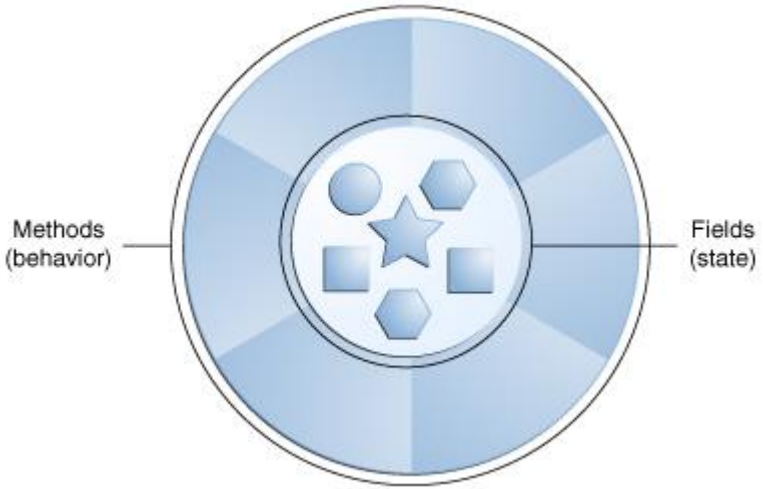
Objects

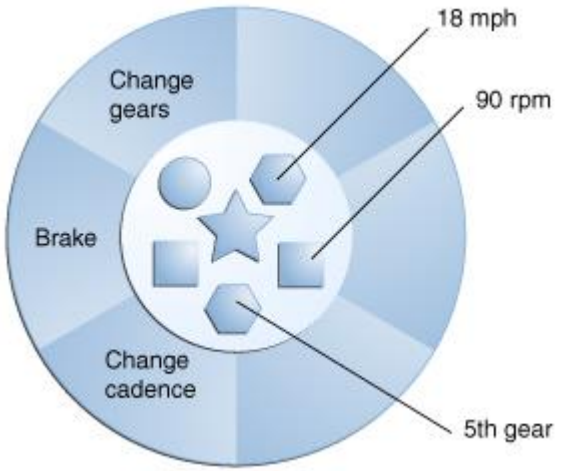
Characteristics of **objects** (real-world or software)

- State
- Behaviour

Why program with objects?

- Modularity
- Information-hiding
- Code re-use
- Pluggability and debugging ease





Java Tutorial "What is an Object?"

Classes vs. objects

Classes are **types**

Objects are **instances of types**

An object is an **instance** of a general **class** of objects

In other words, a class is a **blueprint** from which individual objects are created

Example 1: `boolean` primitive type has instance values **true** and **false**

Example 2: imagine a `Dog` class

Instances: toto, lassie, brianGriffin, scoobyDoo, …

Class declaration in Java

Use the `class` keyword

Give the class a name (use *CamelCase*)

Specify class body inside curly brackets

- Fields (properties)
- Methods (behaviours)

(Optional other things: access modifier(s), superclass, interface(s) – we will address these later in the course)

Example class: bank account

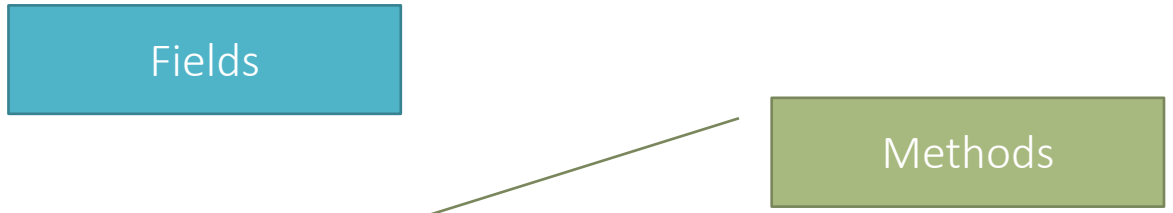
```
class BankAccount {

  int balance;
  String name;

  void deposit(int value) { this.balance += value; }

  void withdraw(int value) { this.balance -= value; }

}
```



Class members: fields and methods

Data fields:

Store state that represent some attributes of the object

For `Dog` class: name, breed, size, age, …

Methods:

Represent behaviour that processes and transforms the object state

For `Dog` class: eat(), sleep(), goForWalk(), …

Special method: `public static void main (String[] args)`

If a class has a `main` method, then you can run it directly (Eclipse: “Run as Java application”)

Fields in Java

Three types of variables:

- Local variables (declared in a method)
- Method parameters (in a method header)
- Member variables in a class – also known as **fields**

Field declarations look the same as local variable declarations, but occur **outside any methods**

An instance variable is accessible in **all methods of a class**

Bank account class revisited

```
class BankAccount {

   int balance;
  String name;

   void deposit(int value) { this.balance += value; }

   void withdraw(int value) { this.balance -= value; }

   BankAccount(String name, int initialAmount) {

      this.name = name;
     this.balance = initialAmount;

   }
}
```

Fields

Methods

Constructor

Constructors

```
BankAccount(String name, int
initialAmount)
{
    this.name = name;
    this.balance = initialAmount;
}
```

Looks like a method with the **same name as the class**

No return type specified (not even `void`)

Sets up initial values for the data fields to initialise object state

Using `this` keyword to refer to current object being created

Use `new` keyword to create a new object:

```
BankAccount b = new BankAccount ("Mary", 0);
```

If no constructor is specified, a default *no-args* constructor is automatically created

No arguments

Sets all fields to their default values (usually 0 or `null`)

Question

For the `BankAccount` class …

How do we stop other code from directly modifying the fields of individual `BankAccount` objects?

```
class BankAccount {
  int balance;
  String name;
  void deposit(int value) { this.balance += value; }
  void withdraw(int value) { this.balance -= value; }
  BankAccount(String name, int initialAmount) {
    this.name = name;
    this.balance = initialAmount;
  }
}
```

```
BankAccount b = new BankAccount("Mary", 1000);
b.value += 1000;
b.name = "Eve";
```

Visibility modifiers

Modifier	Same class	Same package	Any subclass	Any class
`public`	●	●	●	●
`protected`	●	●	●	
(default)	●	●		
`private`	●			

Used to limit the visibility of class members (fields and methods)

Specify as part of member declaration – `private int balance;`

Bank account revisited (again)

```
public class BankAccount {

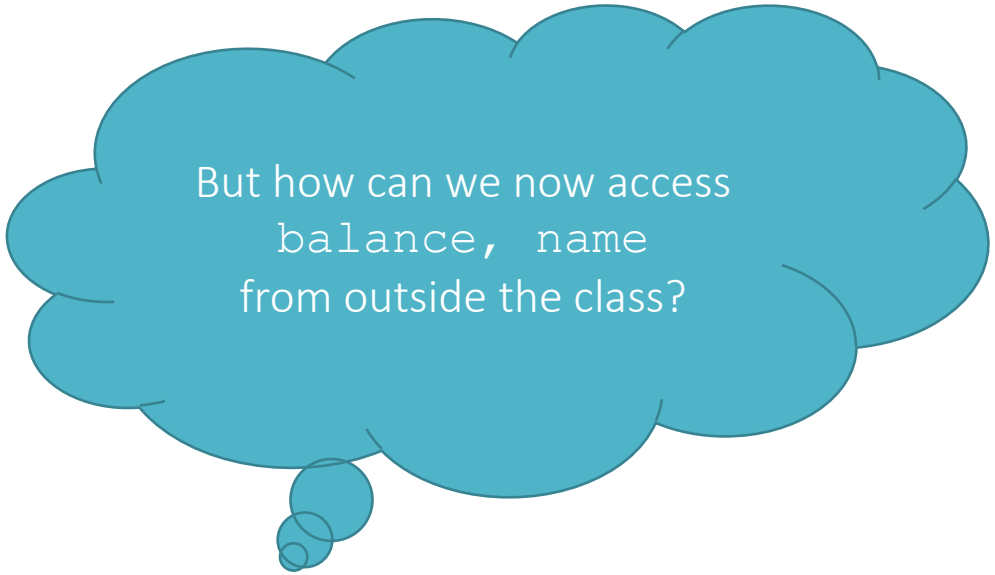
  private int balance;
  private String name;

  public void deposit(int value) { this.balance += value; }

  public void withdraw(int value) { this.balance -= value; }

  public BankAccount(String name, int initialAmount) {

    this.name = name;
    this.balance = initialAmount;
  }
}
```



Getters and setters

Give controlled access to private properties

```
public String getName() {
      return name;
}

public void setName (String name) {
      this.name = name;
}

// … and so on
```

Bank account class again – added ID field

```
public class BankAccount {

   private int balance;
   private String name;
   private int id;

   private static int NEXT_ID = 0;

   public BankAccount(String name, int initialAmount) {

      this.name = name;
      this.balance = initialAmount;
      this.id = BankAccount.NEXT_ID++;

   }
}
```





Static members

Associated directly with the class itself, not with any object of that class

Static field: only one variable, no matter how many objects of the class have been created (even zero)

Examples you have seen: `Double.MAX_VALUE`, `System.out`

Static method: performs a general task for the class; can only access other static members

Examples you may have seen: `Math.random()`, `Integer.parseInt()`

In updated `BankAccount` class, `NEXT_ID` field is static (why?)

Inheritance

Objects (world or software) have some features in common

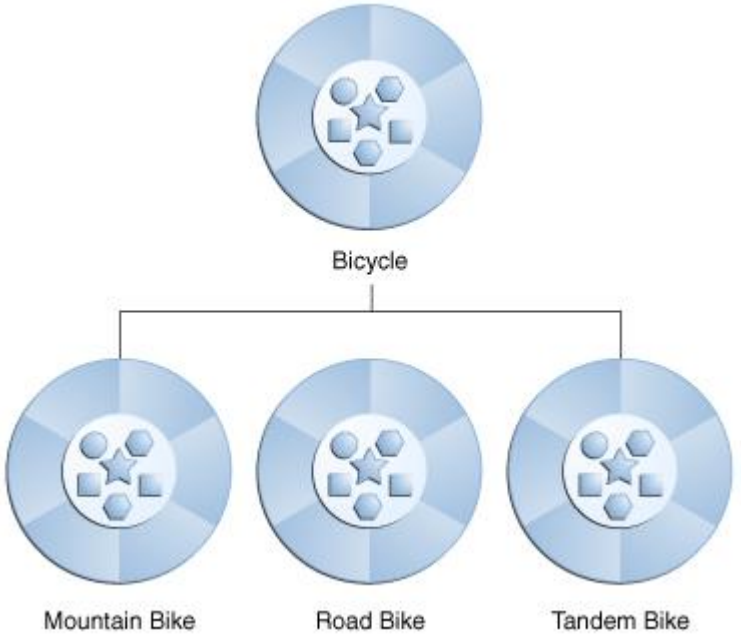
In OO programming, classes can **inherit** state and behaviour (fields and methods) from other classes

- Subclass is a **specialised version** of the superclass

In Java, a class can have **exactly one** superclass

- If superclass isn't specified, then it inherits from `Object`

Subclasses can **override** superclass methods to provide specialised behaviour



Java Tutorial "What is Inheritance?"

Inheritance in Java

```
public class Animal {
    protected String name;

    public void move() {
        System.out.println(name
              +  “can move”);
    }
    public String getName() {
        return this.name;
    }
}
```

```
public class Dog extends Animal {
    private String breed;

    public void move() {
        System.out.println(name
            + “can walk and run”);
    }
}
```

What you can do in a subclass (Fields)

Use the inherited fields just like other fields *(except if they are private)*

Declare a field in the subclass with the same name as in the superclass

This is known as **hiding** the parent field (not recommended!)

Declare new fields that are not in the superclass

What you can do in a subclass (Methods)

Use the inherited methods directly *(unless they are private)*

Override an instance method by writing a new method with the same signature

Hide a static method by writing a new method with the same signature

Declare new methods that are not in the superclass

Method overriding

If a subclass has an instance method with an **identical** signature (name, parameters, return type*) as a parent class …

… then the subclass method **overrides** the parent method

This is a method for specifying alternative (specialised) behaviour for the subclass

```
public class Animal {
    public void move() {
        System.out.println("Animals can move");
    }
}

public class Dog extends Animal {
    public void move() {
        System.out.println("Dogs can walk and run");
    }
}
```

https://www.tutorialspoint.com/java/java_overriding.htm

* Actually, the subclass method's return type could also be a subclass of the parent class method's return type – this is known as a **covariant return type**.

Polymorphism

Literal meaning: “many forms”

In OO design: whenever an instance of class A is expected, you can also use an instance of any subclass of A

If a method is overridden in a subclass, Java will always use the most-specific overridden version

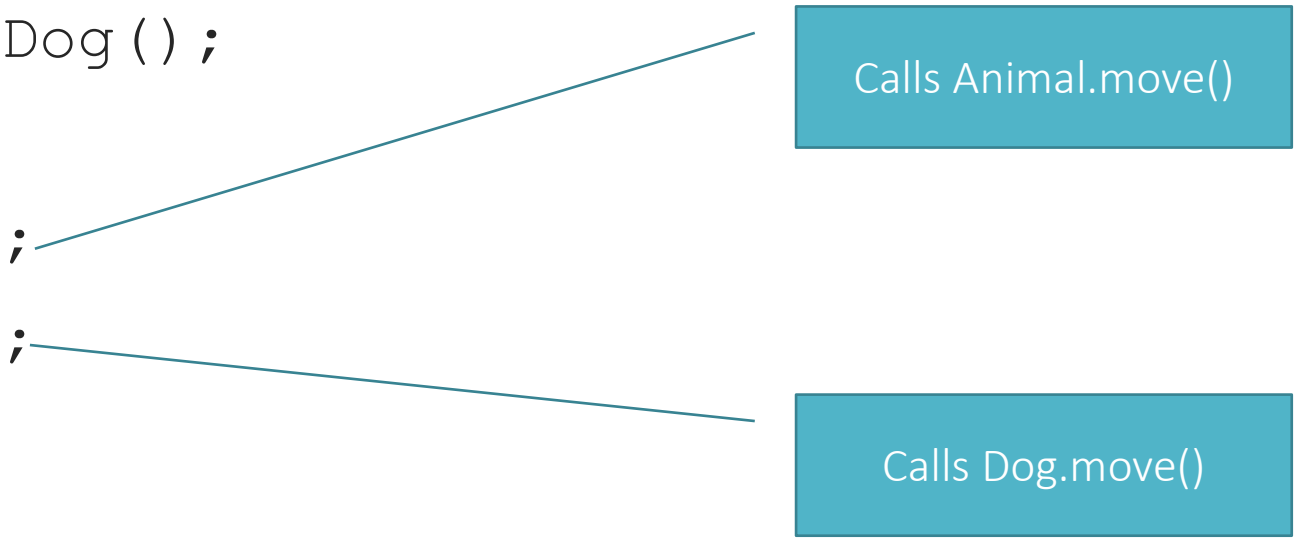
Even when the variable type is the superclass

Supported by **virtual method invocation**: method calls are dynamically dispatched based on the **runtime** type of the receiver object

Polymorphism example

```
Animal a1, a2;
a1 = new Animal();
a2 = new Dog();

a1.move();
a2.move();
```



Built-in methods

All objects inherit some methods from the root class `java.lang.Object`

- `toString()`: generates a `String` representation of the object
- `equals()`: compares two objects for equality and returns a `boolean` result
- `hashCode()`: returns an integer associated with the class for use in more complex data structures

You can override these methods to provide class-specific behaviour

toString() for BankAccount

Default behaviour: prints the class and the memory address of the BankAccount (rarely useful)

Overridden behaviour: prints the information in the relevant bank account fields

```
public String toString() {
    return "Account #" + id
           + " Name=" + name
           + ", balance="
           + balance;
}
```

Default (“no-args”) constructor

If you do not specify a constructor for your class, a default constructor is created

Properties of the default constructor

- Takes no parameters (“no-args”)
- Sets all fields to default values (0 for numeric types, false for Boolean, null for non-primitive types)

This constructor is **only created if you do not specify your own constructor**

```
public Bicycle() {
    this.gear = 0;
    this.cadence = 0;
    this.speed = 0;
}
```

Constructors and inheritance

Recall:

- Constructors look like a method with same name as class; no return type
- If no constructor is specified, a default **no-args** constructor is created

What about inheritance?

- "Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass." *(Java Tutorial)*

Also:

- "If a constructor does not explicitly invoke a superclass constructor, the Java compiler automatically inserts a call to the no-argument constructor of the superclass. If the super class does not have a no-argument constructor, you will get a compile-time error. `Object` does have such a constructor, so if `Object` is the only superclass, there is no problem."

Constructors and inheritance

A subclass **does not inherit** the superclass's constructors

… But can call them from its own constructor using `super`

```
public class MountainBike extends Bicycle {
    public int seatHeight;

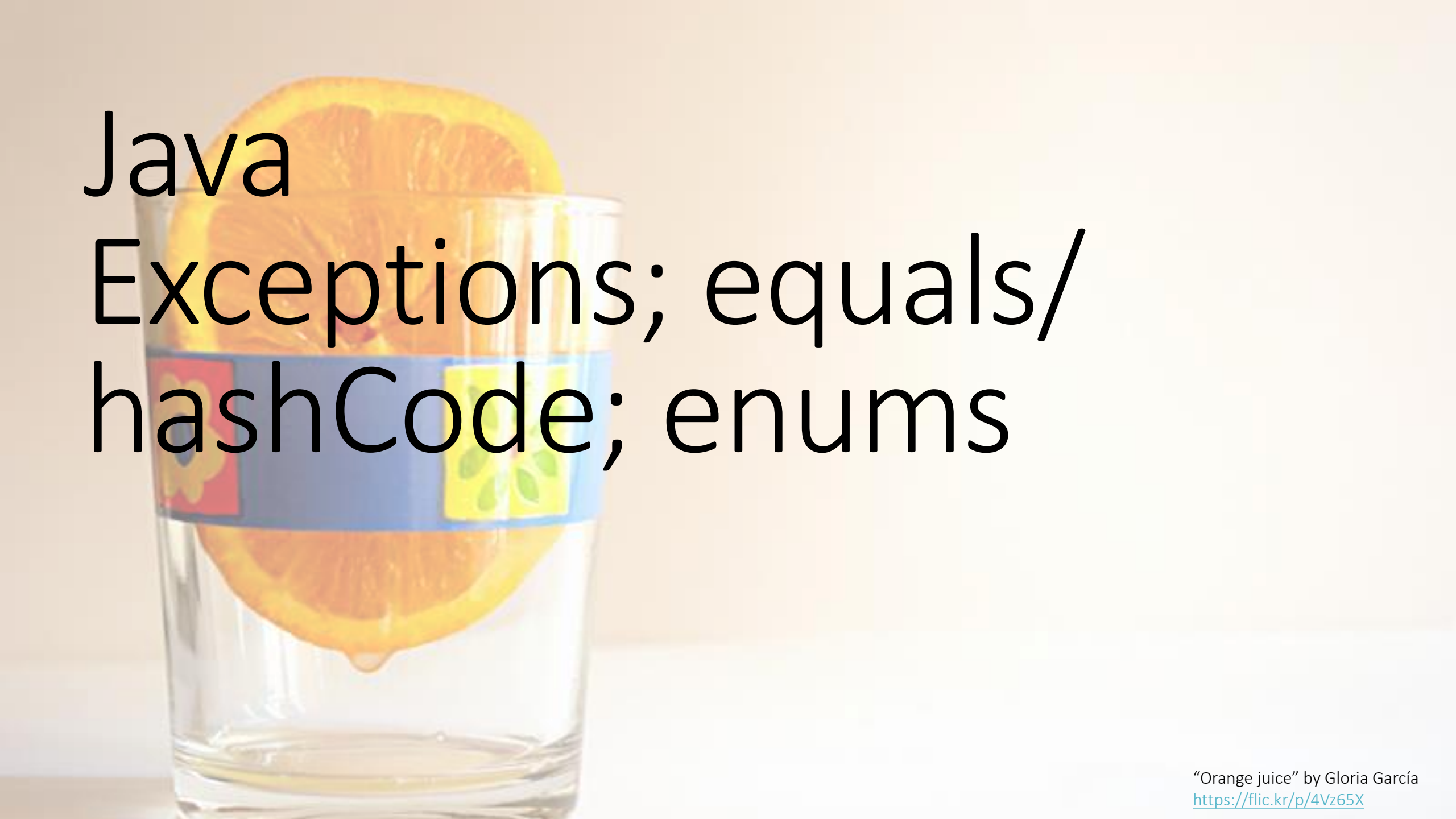
    public MountainBike(int seatHeight, int cadence, int speed, int gear) {
        super(cadence, speed, gear);
        this.seatHeight = seatHeight;
    }
}
```

If this line wasn't here, Java would automatically insert

`super();`

This would not compile!!!

Java Exceptions; equals/ hashCode; enums



"Orange juice" by Gloria García
https://flic.kr/p/4Vz65X

Exceptions

When an error occurs in program execution, an `Exception` is **thrown**

(Exceptions are also Java objects like any other; parent class is `java.lang.Exception`)

Unless the exception is **caught**, the entire program will crash



Checked and unchecked exceptions

UNCHECKED EXCEPTIONS

Do not need to be explicitly handled

- Program will still compile and run without any special handling

Generally indicate **programming/logic bugs** that an application cannot reasonably recover from

Example: `ArrayIndexOutOfBoundsException`

CHECKED EXCEPTIONS

Must be explicitly handled

- Program will not compile unless you deal with them somehow

Generally indicate conditions that a well-written application should anticipate and recover from

Example: `FileNotFoundException`

Handling exceptions #1: Catching

- Wrap a `try {}` block around any code that might throw an Exception
- Must be followed by one (or more) `catch {}` blocks
  - First one whose parameter matches the thrown exception is executed
- Optional `finally {}` block
  - Executed after entire rest of the try block

```
try {
        // code that might
        // throw Exception
} catch (Exception ex) {
        // deal with it
} finally {
        // clean up
}
```

Handling exceptions #2: Passing on

If you do something that might throw an exception, you can add that exception to the `throws` clause of the current method

Then anyone who calls your method will need to handle the exception (by catching or passing on)

```
public void doSomething()
      throws IOException
{
      // code that might
      // throw IOException

}
```

Throwing an Exception

Use the **`throw`** keyword:

```
throw new Exception ("Invalid input");
```

You can throw an Exception at any point in your code

String parameter indicates the message (available through `ex.getMessage()`)

If you throw a checked Exception, you also need to add it to the header of your method with the **`throws`** keyword

```
public String processInput (String input) throws Exception { … }
```

Advantages of using Exceptions

1. Separating out error-handling code
   Instead of a series of if/then/else statements
   Just “assume” that things will work and deal with errors elsewhere
2. ***Propagating*** errors up the call stack
   i.e., sending errors along until they reach a method that is prepared to handle them
3. Grouping error types
   Exception is a class, and can be subclassed
   => different types of Exceptions can be conceptually grouped together
   (I/O exceptions, for example)

Methods of `java.lang.Object`

```
protected Object clone()
boolean equals (Object obj)
protected void finalize()
public Class<?> getClass()
public int hashCode()
public void notify() / notifyAll()
public String toString()
public void wait() / wait(long timeout) / wait(long timeout, int nanos)
```

java.lang.Object.equals() documentation

Indicates whether some other object is “equal to” this one

The `equals` method implements an equivalence relation on non-null object references:

- It is ***reflexive***: for any non-null reference value `x`, `x.equals(x)` should return `true`.
- It is ***symmetric***: for any non-null reference values `x` and `y`, `x.equals(y)` should return `true` if and only if `y.equals(x)` returns `true`.
- It is ***transitive***: for any non-null reference values `x`, `y`, and `z`, if `x.equals(y)` returns `true` and `y.equals(z)` returns `true`, then `x.equals(z)` should return `true`.
- It is ***consistent***: for any non-null reference values `x` and `y`, multiple invocations of `x.equals(y)` consistently return `true` or consistently return `false`, provided no information used in `equals` comparisons on the objects is modified.
- For any non-null reference value `x`, `x.equals(null)` should return `false`.

Default implementation of equals()

"The most discriminating possible equivalence relation on objects"

Returns true **if and only if** x and y refer to the **same** object (i.e., x == y is true)

Gives the correct result for primitive types (int, double, char, etc.)

Does not check if objects are **equivalent** – i.e., if their contents are the same

```
ArrayList<Integer> l1 = new ArrayList<>();
l1.add (1);
ArrayList<Integer> l2 = new ArrayList<>();
l2.add (1);
boolean result = l1.equals(l2); // Default would return false
```

java.lang.Object.hashCode() documentation

Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by `HashMap`.

The general contract of `hashCode` is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the `hashCode` method must consistently return the same integer, provided no information used in `equals` comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.
- If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result.
- It is not required that if two objects are unequal according to the `equals(java.lang.Object)` method, then calling the `hashCode` method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hash tables.

Default implementation

"As much as is reasonably practical, the `hashCode` method defined by class `Object` does return distinct integers for distinct objects. (This is typically implemented by converting the internal address of the object into an integer, but this implementation technique is not required by the Java™ programming language.)"

equals() and hashCode()

"If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects **must produce the same integer result**."

So if you override `equals()`, **you must also override `hashCode()`**

Enumerated types

An enum type is a special data type that allows a variable to be one of a set of predefined constants

Common examples:

- Compass directions (NORTH, SOUTH, EAST, WEST)
- Days of week, months of year, etc.

Declaring an enum in Java

```
public enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY,
  }
```

Note: values are constants ==> conventionally written in ALL_CAPS

You use the `enum` keyword **instead of** `class`

An enum called Day should be in a class Day.java

An enum is a special class

It has **methods**

- Built-in static method values() that returns an array of all values
- Built-in static method valueOf() that parses a string into an enum constant
- Appropriate definitions of compareTo(), equals(), hashCode(), toString()
- Other methods:
  - ordinal() -- returns the position of this constant in the list
  - name() -- returns the name of this constant
- Any other methods that you define

You can define **fields** as well if necessary

Java
Abstract and final classes, interfaces



Image: “Serious orange juicing” by Leonid Mamchenkov
https://flic.kr/p/xXdD42

Solution: the `final` keyword

If a method is marked as `final` then it **cannot be overridden**

- Provides predictable behaviour
- Especially relevant where method has security implications

If a class is marked as `final` then it **cannot be subclassed**

- Particularly useful for **immutable** classes such as `String` or `Double`
- … Or if all methods would require `final`

`final` fields, parameters, and variables

If a **field** is declared `final`, then its value can never be changed

- Value can only be set at declaration time or in a constructor

If a **parameter** is declared `final`, then its value can never be changed inside the method

If a **variable** is declared `final`, then its value can never be changed

- Value can be set at declaration or later, but can never be changed thereafter

```
public class Test {

        private final int field1 = 1;
        private final int field2;

        public Test (final int arg) {

                this.field2 = arg; // okay
                this.field1 = 5;   // error

                arg = 3;           // error

                final int foo;     // okay
                final int bar = 2; // okay

                foo = 3;           // okay
                foo = 4;           // error
                bar = 4;           // error
        }
}
```

What about `static final`?

Generally used to define **constants**

- `final` modifier means that the value cannot change
- Constant names are (usually) written in `ALL_CAPS`

Examples:

- `Math.E` *The double value that is closer than any other to e, the base of the natural logarithms*
- `Long.MAX_VALUE` *A constant holding the maximum value a `long` can have, 2^{63}-1*
- `System.out` *The "standard" output stream*

Abstract classes and methods

Some classes have “holes” in them – methods that **must** be overridden in subclasses

Such classes are marked as `abstract`

Methods that must be overridden are marked as `abstract` too

If a subclass does not implement all `abstract` methods, it must also be marked `abstract`



First abstract watercolor, painted by Wassily Kandinsky, 1910.

Example

```
public abstract class TwoDimensionalPoint {
  protected double x;
  protected double y;

  public abstract double distanceToOrigin();
}

 public class CartesianPoint extends TwoDimensionalPoint {

   public double distanceToOrigin() {
     return Math.sqrt(x*x+y*y);
   }
 }

 public class ManhattanPoint extends TwoDimensionalPoint {

   public double distanceToOrigin() {
     return Math.abs(x) + Math.abs(y);
   }
 }
```

This method ensures that all subclasses meet a given API

- In the example, all subclasses of `TwoDimensionalPoint` must implement `distanceToOrigin()`

But: it doesn't make sense to implement `distanceToOrigin()` in the superclass

More on abstract methods/classes

Abstract methods do not have a body – just the signature followed by semicolon

```
public abstract double distanceToOrigin();
```

Abstract classes can still have

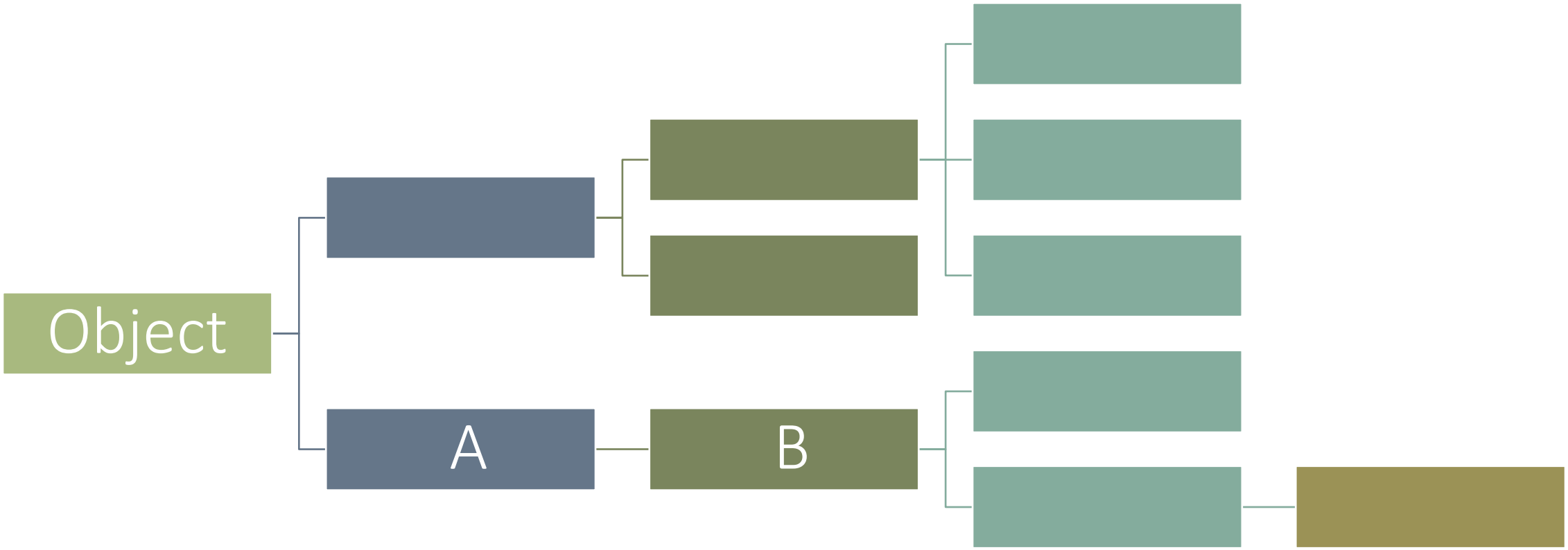
- Constructors
- Fields
- Normal (non-abstract) methods
- Static fields and methods

You **cannot** create instances of abstract classes – only **concrete** subclasses

(Opposite of abstract)

~~`TwoDimensionalPoint p= new TwoDimensionalPoint();`~~

Inheritance in Java

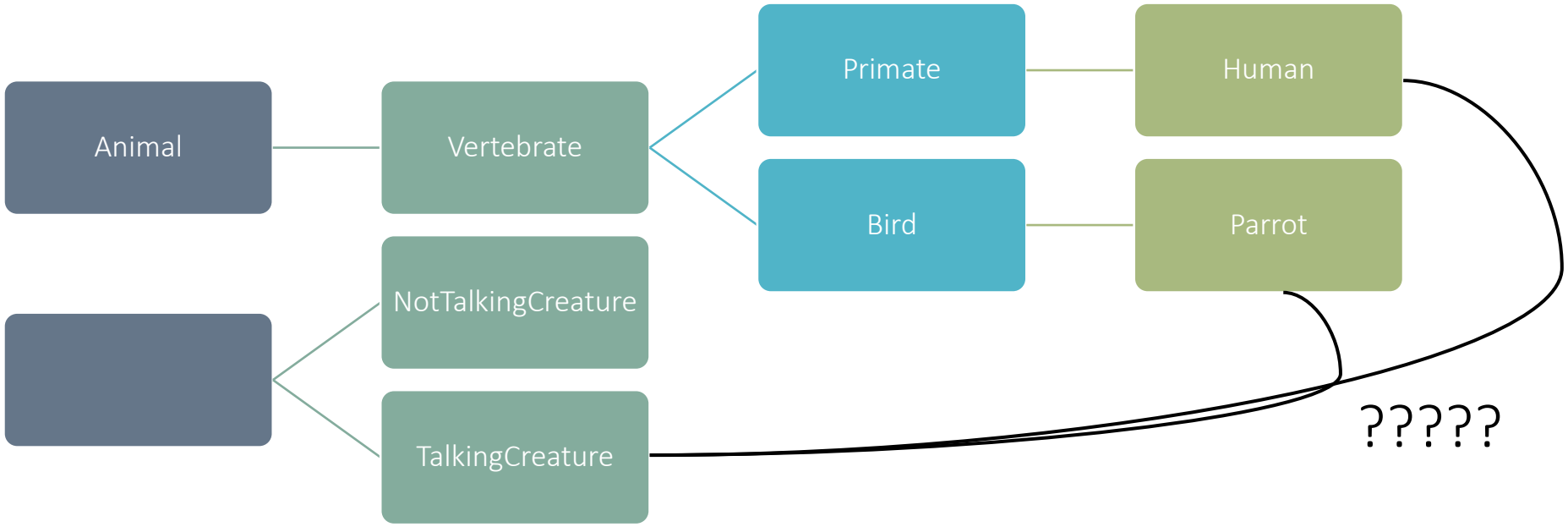


What about *multiple* inheritance?

Example:

A `Person` is both a `Primate` and a `TalkingCreature`

A `Parrot` is both a `Bird` and a `TalkingCreature`



Interface overview

Interfaces represent class relationships **outside the main inheritance hierarchy**

A class can implement any number of interfaces (including zero)

An interface specifies a public API

- Implementation is irrelevant – method signatures only

It is a **contract** that all implementing classes must honour

Interfaces in Java

Similar to a class, but …

- Declared with `interface` keyword
- All (non-`static`) methods are implicitly `public abstract`
- All fields are implicitly `public static final`
  - *i.e., constants*
- Has no* instance-level fields or methods
  - *Eliminates issues with multiple inheritance of state and implementation*

```
public interface TalkingCreature {

    void speak(String s);

}
```

```
public interface List {
    int size();
    boolean isEmpty();
    boolean contains (Object o);
    boolean remove (Object o);
    void clear();
    // …
}
```

* Except for default methods (introduced in Java 8) … see https://docs.oracle.com/javase/tutorial/java/IandI/defaultmethods.html for details

Implementing an interface

- Use `implements` keyword
  - Conventionally, comes after `extends`
- Provide a definition for all methods declared in each interface
  - … or else declare class as `abstract`
  - All method implementations must be `public`
- Classes can implement multiple interfaces (comma-separated)

```
public class Person
    extends Primate
    implements TalkingCreature {

    public void speak (String s) {
        // …
    }

}

public class Parrot
    extends Bird
    implements TalkingCreature {

    public void speak (String s) {
        // …
    }

}
```

Using an interface as a type

You can use an interface name anywhere you use any other data type name

- Variable declarations
- Method parameters
- etc

You cannot directly create an instance of an interface (through `new`)

```
TalkingCreature c = new Person();

public void listen (TalkingCreature c)
{
    // …
}
```

Interface vs. abstract class

INTERFACE

- Cannot be instantiated
- Has no constructor
- All methods are public
- Contain only constant fields
- Classes can implement **multiple** interfaces

ABSTRACT CLASS

- Cannot be instantiated
- Has a constructor
- Methods can have any access modifier
- Contain constant and “normal” fields
- Classes can have **at most one** parent class



"Venus Williams Opens Jamba Juice At Dupont Circle" by www.glynlowe.com https://flic.kr/p/cuMMw3

Java
Packages, Collections

Packages

Package: groups together related resources (usually classes)

Why put code in a package?

- Makes it obvious that types are related
- Makes it possible to find types for a specific purpose (given good package name)
- Type names won't conflict with names from other packages
- Types within package can have unrestricted access to one another
  - *While restricting access for types outside the package*

Creating a package

Choose a name

Put a `package` statement at the top of every source file for that package

```
package my.package.name;
```

Ensure that all source files are in a directory corresponding to the package name

If you don't use a `package` statement, then all files are in the default package

Accessing package members

To use a public type from outside the package, you need to **import** it

```
// At top of source file, after package statement
import java.util.Scanner;
// … later on, inside the class …
Scanner stdin = new Scanner(System.in);
```

Java Collections framework

A standard set of built-in classes for representing and manipulating collections

Each Collections class groups **related elements** into a **single unit**

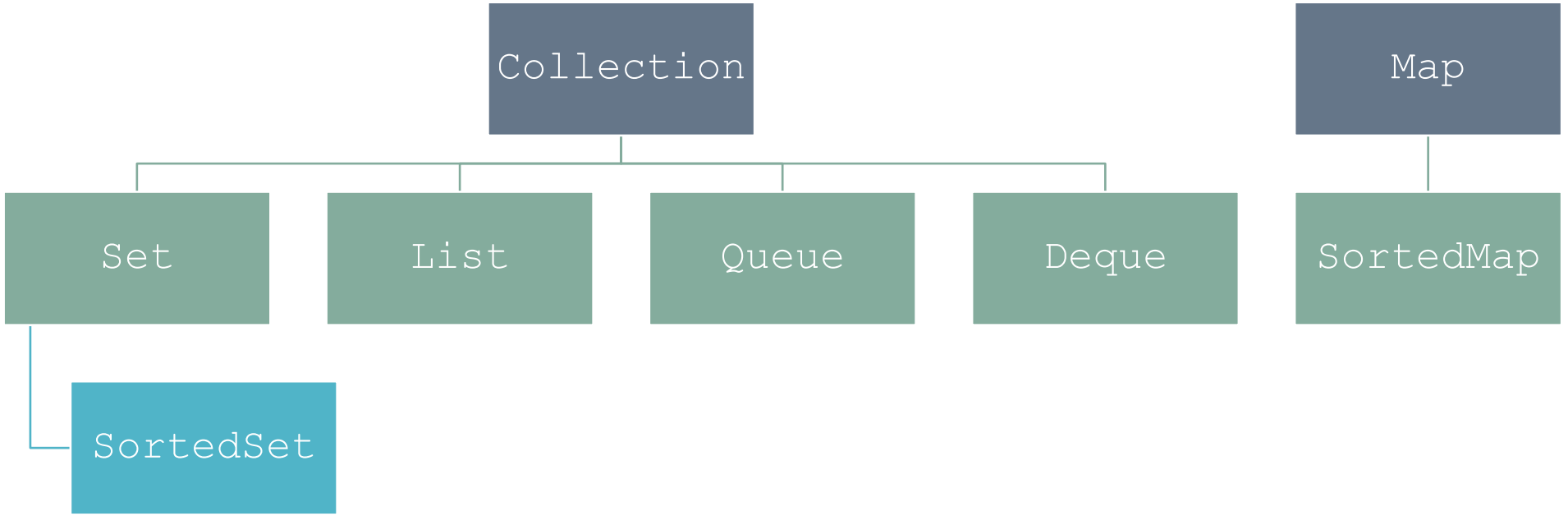
Examples:

- **ArrayList** – acts like a variable-length array
- **HashSet** – a group of unique elements
- **Stack** – a list with last-in/first-out semantics
- **HashMap** – a **dictionary** (e.g., a telephone directory)

Structure of Collections framework

Base class:* `java.util.Collection`

Methods: `add(), remove(), contains(), size(), toArray()`



** Actually, everything in this picture is an interface*

Advantages of Collections

Reduces programming effort by providing pre-written data structures and algorithms

Increases performance by providing high-performance implementations
    Implementations are interchangeable – can switch to tune performance

Provides interoperability by allowing Collections to be passed back and forth

Reduces effort to learn new APIs by providing a common interface

Reduces effort to design APIs by giving design specifications

Fosters software reuse by providing a standard interface

(List adapted from https://docs.oracle.com/javase/tutorial/collections/intro/index.html)

java.util.ArrayList

A Collections class (specifically, a `List`) that implements **variable-length** arrays

More flexible than built-in arrays, but less efficient

Acts as a wrapper around an underlying array that grows and shrinks dynamically

`ArrayList` is a **class** – so elements are added and removed by **methods**
(Not by built-in Java syntax as with normal arrays)

It has a **capacity** (size of internal array) and a **size** (number of elements in the list)

Capacity is increased when necessary – purely internal

Size is increased/decreased as elements are added and removed, and checked for operations

In general: IndexOutOfBoundsException if `(index < 0 || index >= size())`

Array vs ArrayList at a glance

Operation	Array	ArrayList
Declaration	`String[] strings;`	`ArrayList<String> strings;`
Initialisation	`strings = new String[10];`	`strings = new ArrayList<>(10);`
Setting element	`strings[5] = "foo";`	`strings.set(5, "foo");`
Accessing element	`String s = strings[5];`	`String s = strings.get(5);`
Getting size	`int n = strings.length;`	`int n = strings.size();`
Adding element	*n/a*	`strings.add("foo");` `strings.add(5, "foo");`
Removing element	*n/a*	`strings.remove("foo");` `strings.remove(5);`

Finding element	`Arrays.binarySearch(strings, "foo");`	`strings.contains("foo");`
		`strings.indexOf("foo");`
		`strings.lastIndexOf("foo");`

`List` vs `ArrayList`?

`List` is the **high-level Collection type** (actually it's an interface)
- Specifies methods including **add**, **clear**, **isEmpty**, **remove**, **set**, …

`ArrayList` is the **specific type of List**
- Provides a concrete implementation
- Additional methods related to capacity

When to use which?
- Use `ArrayList` …
  - *When initialising a new variable*
  - *If you want to manipulate capacity*
- Use `List` all other times – allows implementations to be swapped cleanly

Generic types




```
List<String> strList = new ArrayList<>();
```

Collection classes are **type-parameterised**

- The type specified in angle brackets after the name specifies the type of the elements stored in that Collection
- If you don't specify any type, then it will use `java.lang.Object`

(Polymorphism: subclasses of specified type will also be accepted)

Generic types were added to Java in Java 1.5 (2004)

Why use generic types?

Compile-time error checking

```
List<String> strList = new ArrayList<>();
strList.add ("foo");
strList.add (new java.util.Scanner()); // fail
```

Iteration can be much cleaner (especially with new-style iteration)

```
for (String s : words) {
    System.out.println (s.toLowerCase());
}
```

```
for (int i = 0; i < words.size(); i++) {
    String s = (String)words.get(i);
    System.out.println (s.toLowerCase());
}
```

Primitive types and generics

The <*type*> generic parameter needs to be a **class**

Primitive types will not work!

~~`List<int> intList;`~~

Solution: Use **wrapper** classes (`int/Integer, long/Long`, etc.)

```
List<Integer> intList = new ArrayList<>();
```

But you don't want to have to do this all the time …

```
Integer i2 = new Integer (i);
intList.add (i2);
int value = intList.get(5).intValue();
```

Boxing and unboxing

Good news: Java **automatically** converts between wrapper classes and primitive types

(Also since Java 1.5)

```
List<Integer> intList = new ArrayList<>();
intList.add (5);
intList.add (10);
int value = intList.get (0);
Integer value2 = intList.get(1) * 100;
```

Sets

Interface: `java.util.Set`

- Concrete implementations: `HashSet, TreeSet, LinkedHashSet`

Differences to List

- Cannot contain duplicate elements
  - *add() method enforces this – returns true/false indicating if element was already in set*
- Two sets are equal if they contain the same elements, regardless of implementation

Maps

Interface: `java.util.Map`

- Concrete implementations: `HashMap, TreeMap, LinkedHashMap`

Provides a mapping from keys to values

- Cannot contain duplicate keys
- Each key maps to exactly one value

Useful methods:

- `get(key)` – return the value associated with a key (null if no value)
- `put(key, value)` – set the new value associated with that key